HapticNV MCU Software Porting Guide

1 Software Overview

Driver File	haptic_nv.c,haptic_nv.h,haptic_nv_reg.h,haptic_nv_params.c,haptic_nv_config.c, aw862x.c,aw862xx.c,aw8623x.c
IC Support	AW862X:aw8623 aw8624 AW862XX:aw86214 aw86223 aw86224 aw86225 AW8623X:aw86233 aw86234 aw86235 AW8624X:aw86243 aw86245
I²C Address	0x5A/0x58

2 Software Porting

1. Open the haptic_nv_config.c file in the driver package and implement the functions in the file based on your own platform
2. Open the driver package aw862x.c/aw862xx.c/aw8623x.c/aw8624x.c file, using the motor nominal f0 and rated voltage according to modify the dts f0_pre and cont_lra_vrms parameters of the structure
3. Put all *.c and *. h files into the Awinic folder in the project directory
4. Right-click "Project", create "Awinic" module, add aw862x.c, aw862xx.c, aw8623x.c, aw8624x.c haptic_nv.c，haptic_nv_config.c，haptic_nv_params.c
5. Open haptic_hv.h, select the chip macro control AW862X_DRIVER/ AW862XX_DRIVER/ AW8623X_DRIVER/ AW8624X_DRIVER according to the chip used, reduce the amount of code, and save platform resources
6. Open haptic_nv.h, and enable and disable AW_RTP_AUTOSIN macro control according to whether RTP autosin function is used (disabled by default).
7. Open haptic_nv.h, AW_IRQ_CONFIG config using interrupt, otherwise use trig function, AW_CHECK_RAM_DATA config ram read back check, AW_ENABLE_RTP_PRINT_LOG open rtp play process detailed log, HAPTIC_NV_DOUBLE is configured for use with dual motors
8. Add #include "haptic_nv.h" to main.c, configure the I2C and timer (hi2c1 , hi2c2 and htim3 are used by default for the driver), and compile the project

3 Validation Of Driver Migration

Chip identification is successful:

```
[haptic_nv][0749]haptic_nv_boot_init: haptic nv driver version v0.2.0
[haptic_nv][0476]parse_chipid: detected aw8624.
[haptic_nv][0442]sw_reset: enter!
```

```
[haptic_nv][0749]haptic_nv_boot_init: haptic_nv driver version v0.2.0
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id
[haptic_nv][0091]haptic_nv_read_chipid: reading chip id
[haptic_nv][0466]parse_chipid: try to replace i2c addr [(0x58)] to read chip id again
[haptic_nv][0487]parse_chipid: aw86224 or aw86225 detected
[haptic_nv][0442]sw_reset: enter!
```

```
[haptic_nv][0582]parse_chipid: aw86233 detected
[haptic_nv][0521]sw_reset: enter!
[haptic_nv][0768]aw8623x_interrupt_setup: enter
```

```
[I][haptic_nv]parse_chipid: try to replace i2c addr [(0x58)] to read chip id again
[I][haptic_nv]parse_chipid: aw86243 detected
```

4 Function And Interfaces

The Haptic_NV MCU compatible driver defines a function pointer structure. After calling the initialization function haptic_nv_boot_init to complete startup initialization, the user can call the function interface through the structure pointer g_func_haptic_nv, as follows：

```
struct aw_haptic_func
{
int (*check_qualify)(void);                          //Chip mass production verification
int (*get_irq_state)(void);                          //Get the interrupt status
int (*get_f0)(void);                                 //Get motor frequency f0
int (*offset_cali)(void);                            //Start offset calibration
void (*trig_init)(void);                             //trig initialization
void (*irq_clear)(void);                             //Read clear interrupt
void (*haptic_start)(void);                          //Start playing
void (*play_stop)(void);                             //Stop playing
void (*cont_config)(void);                           //Register configuration in conT mode
void (*play_mode)(uint8_t);                          //Select the playback mode
void (*ram_init)(aw_bool);                           //RAMINIT enable control
void (*misc_para_init)(void);                        //Initialize the boot register
void (*interrupt_setup)(void);                       //Interrupt the configuration
void (*vbat_mode_config)(uint8_t);                   //In vbat mode
void (*protect_config)(uint8_t, uint8_t);            //Motor protection configuration
void (*calculate_cali_data)(void);                   //Calculate the f0 calibration value
void (*set_gain)(uint8_t);                           //Set the gain
void (*set_wav_seq)(uint8_t, uint8_t);               //Set the waveform
void (*set_wav_loop)(uint8_t, uint8_t);              //Set the playback times
void (*set_rtp_autosin)(uint8_t);                    //Set rtp autosin enable switch
void (*set_rtp_data)(uint8_t *, uint32_t);           //Write RTP data
void (*set_fifo_addr)(void);                         //Set the FIFO address
void (*get_fifo_addr)(void);                         //Get the FIFO address
void (*set_ram_data)(uint8_t *, int);                //Write ram data
void (*get_ram_data)(uint8_t *, int);                //Read ram data
void (*set_ram_addr)(void);                          //Set the RAM address
void (*set_repeat_seq)(uint8_t);                     //Set waveform playback to repeat
void (*set_base_addr)(void);                         //Set the base address
void (*set_trim_lra)(uint8_t);                       //Set the TRIM_LRA register
void (*set_rtp_aei)(aw_bool);                        //Set fast air break
void (*get_vbat)(void);                              //Check the battery voltage
void (*get_reg)(void);                               //Get register data
void (*get_lra_resistance)(void);                    //Motor impedance detection
uint8_t (*get_glb_state)(void);                      //Get the value of the global status register
uint8_t (*judge_rtp_going)(void);                    //Check whether RTP is being played
uint8_t (*rtp_get_fifo_afs)(void);                   //Check whether the FIFO is full
void (*f0_show)(void);                               //Get motor real F0
void (*ram_show)(void);                              //Read back to display RAM waveform data
```

```
void (*cali_show)(void);                                    //Get the calibrated motor F0
void (*irq_handle)(void);                                   //Interrupt handler function
void (*get_ram_num)(void);                                  //Get the number of RAM waveforms
void (*rtp_vib_work)( uint8_t);                             //RTP playback sample function
void (*set_hw_irq_status)(uint8_t);                         //Set the interrupt state
uint8_t (*get_hw_irq_status)(void);                         //Get the interrupt status
int (*f0_cali)(void);                                       //F0 calibration
int (*rtp_going)(void);                                     //RTP play
int (*long_vib_work)(uint8_t, uint32_t);                    //Long shock playback example function
int (*short_vib_work)(uint8_t, uint8_t, uint8_t);           //Short shock playback example function
int (*dual_short_vib)(uint8_t, uint8_t, uint8_t, uint8_t, uint8_t, uint8_t);// Double motor short shock
int (*dual_long_vib)(uint8_t, uint8_t, uint8_t, uint8_t, uint32_t);// Double motor long shock
};
```

5 Single Motor Function Call Reference

1. Initialization process:

Call the initialization function haptic_nv_boot_init() to initialize the haptic chip. If 0 is returned, the initialization is successful. Other interfaces can be used only after the initialization is complete.

2. Short vibration call example:
   g_func_haptic_nv ->short_vib_work(1, 0x80, 1);
3. Long vibration call example:
   g_func_haptic_nv ->long_vib_work(4, 0x80 , AW_MS_UINT*500);
4. RTP playback example:
   g_func_haptic_nv ->rtp_vib_work(0x80);
5. F0 calibration:
   g_func_haptic_nv ->f0_cali();
6. F0 reading after calibration:
   g_func_haptic_nv ->cali_show();
7. Real F0 read:
   g_func_haptic_nv ->f0_show();
8. Motor impedance detection:
   g_func_haptic_nv ->get_lra_resistance();
9. Battery voltage detection:
   g_func_haptic_nv ->get_vbat();

6 Dual Motor Function Call Reference

1. Initialization process:

```
haptic_nv_change_motor(MOTOR_L);                    //Switch control left motor
haptic_nv_boot_init();                              //Left motor chip initialization
haptic_nv_change_motor(MOTOR_R);                    //Switch control right motor
haptic_nv_boot_init();                              //Right motor chip initialization
```

2. Short vibration call example:

```
g_func_haptic_nv->dual_short_vib(1 , 0x80 , 1 , 1 , 0x80 , 1);      //Short vibration of double motors
```

3. Long vibration call example:

```
g_func_haptic_nv->dual_long_vib(4 , 0x80 , 4 , 0x80 , AW_MS_UINT*500);  //Long vibration of double motors
```

4. F0 calibration:

```
haptic_nv_change_motor(MOTOR_L);                    // Switch control left motor
g_func_haptic_nv->f0_cali();                        //Left motor F0 calibration
haptic_nv_change_motor(MOTOR_R);                    // Switch control right motor
g_func_haptic_nv->f0_cali();                        // Right motor F0 calibration
```

5. F0 reading after calibration:

```
haptic_nv_change_motor(MOTOR_L);                    // Switch control left motor
g_func_haptic_nv->cali_show();                      //Left motor F0 after calibration
haptic_nv_change_motor(MOTOR_R);                    // Switch control right motor
g_func_haptic_nv->cali_show();                      // Right motor F0 after calibration
```

6. Real F0 read:

```
haptic_nv_change_motor(MOTOR_L);                    // Switch control left motor
g_func_haptic_nv->f0_show();                        // Left motor real F0
haptic_nv_change_motor(MOTOR_R);                    // Switch control right motor
g_func_haptic_nv->f0_show();                        // Right motor real F0
```

7. Motor impedance detection:

```
haptic_nv_change_motor(MOTOR_L);                    // Switch control left motor
g_func_haptic_nv-> get_lra_resistance ();           // Left motor impedance detection
haptic_nv_change_motor(MOTOR_R);                    // Switch control right motor
g_func_haptic_nv-> get_lra_resistance ();           // Right motor impedance detection
```

8. Battery voltage detection:

```
g haptic_nv_change_motor(MOTOR_L);                  // Switch control left motor
g_func_haptic_nv-> get_vbat();                      // Left motor voltage detection
haptic_nv_change_motor(MOTOR_R);                    // Switch control right motor
g_func_haptic_nv-> get_vbat();                      // Right motor voltage detection
```

7 F0 Calibration Instructions

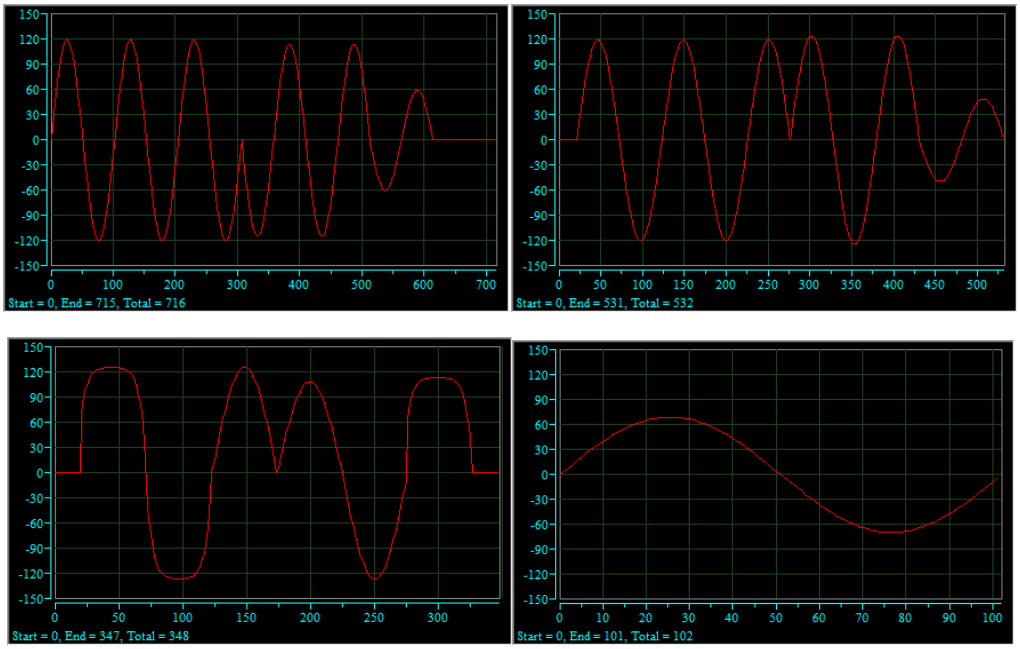
Two interfaces for saving and obtaining F0 calibration values are reserved in the haptic_hv_config.c, which need to be implemented by the customer:

1. void haptic_nv_set_cali_to_flash(void) // Write the calibration value into flash to save
2. void haptic_nv_get_cali_from_flash(void) // Read the calibration value from flash and update it to the driver

If the AW_F0_CALI_DURING_STARTUP macro is defined, the default device performs F0 calibration during startup, and the above two interfaces will not be used

8 Waveform Examples

Reference RAM waveforms(Wave1 ~ Wave3 are short seismic waveforms and Wave4 is long seismic specified waveforms.):



9 Precautions

1. RTP playback needs to be configured with external interrupt IRQ_PIN and falling edge trigger mode: when detecting hardware interrupt trigger, jump interrupt function processing;
2. In RTP mode, the i2c must be used independently by Haptic;
3. The driver needs to invoke the haptic_nv_boot_init function to initialize the g_func_haptic_nv pointer;
4. RTP is not supported by dual motor drive at present.

Revision History

Version	Date	Description
V1.0	2021-11-22	Initial release
V1.1	2022-01-27	1. Add the description of haptic_nv_config.c and improved driver adaptation steps
V1.2	2022-06-06	1. Key information is marked in red 2. Add migration validation 3. Add a watermark
V1.3	2022-08-05	1. Add aw8623x 2. Delete get_first_addr interface 3. Delete AW_LOG_CTRL
V1.4	2022-08-18	1. Add dual motor support
V1.5	2023-02-14	1. Delete superfluous *.h 2. Replacing a document template
V1.6	2024-02-05	1. Add AW_RTP_AUTOSIN macro control description. 2. Add aw8624x support.

Declaration

All trademarks are the property of their respective owners. Information in this document is believed to be accurate and reliable. However, Shanghai AWINIC Technology Co., Ltd (AWINIC Technology) does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information.

AWINIC Technology reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. Customers shall obtain the latest relevant information before placing orders and shall verify that such information is current and complete. This document supersedes and replaces all information supplied prior to the publication hereof.

AWINIC Technology products are not designed, authorized or warranted to be suitable for use in medical, military, aircraft, space or life support equipment, nor in applications where failure or malfunction of an AWINIC Technology product can reasonably be expected to result in personal injury, death or severe property or environmental damage. AWINIC Technology accepts no liability for inclusion and/or use of AWINIC Technology products in such equipment or applications and therefore such inclusion and/or use is at the customer’ s own risk.

Applications that are described herein for any of these products are for illustrative purposes only. AWINIC Technology makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

All products are sold subject to the general terms and conditions of commercial sale supplied at the time of order acknowledgement.

Nothing in this document may be interpreted or construed as an offer to sell products that is open for acceptance or the grant, conveyance or implication of any license under any copyrights, patents or other industrial or intellectual property rights.

Reproduction of AWINIC information in AWINIC technical document is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. AWINIC is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of AWINIC components or services with statements different from or beyond the parameters stated by AWINIC for that component or service voids all express and any implied warranties for the associated AWINIC component or service and is an unfair and deceptive business practice. AWINIC is not responsible or liable for any such statements.

